



CampusPulse-Shield

A Chained-Vulnerability Web Application

Break it on purpose. Patch it on principle.

Course: Cyber Security · **Project:** Topic 5 — Chained Vulnerabilities

Submitted to

Eng. Lobna & Eng. Eman

By

Nayrouz Ahmed — 221011969

Mariam Ashraf — 221002547

Youssef Negm — 221011914

Mohamed Eldairouty — 221001719

Setting the Scene

Every university portal we've ever used has at least one familiar feature: a noticeboard, a course catalogue, a grade page that students reload obsessively right before finals. For our project we wanted to take exactly that kind of app — the kind built and trusted daily by students and faculty — and ask a simple uncomfortable question: *what if it was built carelessly?*

CampusPulse-Shield is the answer. It's a small Flask-based student portal — accounts, courses, professors, announcements, enrollments, grades — packaged as **two parallel deployments**:

- **VULNERABLE :5000** — contains three real, exploitable security flaws, chained end-to-end.
- **HARDENED :5001** — same UI, same features, every flaw fixed with its textbook mitigation.

The two builds run side-by-side. The attack we'll demonstrate lands on the red one and dies on the green one — without us changing a single character of the attack payload. That contrast is the whole point of the project.

Goal of this report: document the architecture we built, the tools we used, and the precise logic behind each mitigation — so that a reviewer reading these pages can reconstruct both the attack and the defence from the diagrams and screenshots.

The Application — A Quick Tour

Before we get to the attack itself, here's a thirty-second look at what we actually built. CampusPulse-Shield is a fully working multi-role portal: a student can register, log in, view their enrolled courses, check grades, and post on a community feed; a TA can manage student enrolments; an admin owns the catalogue and the grade book.

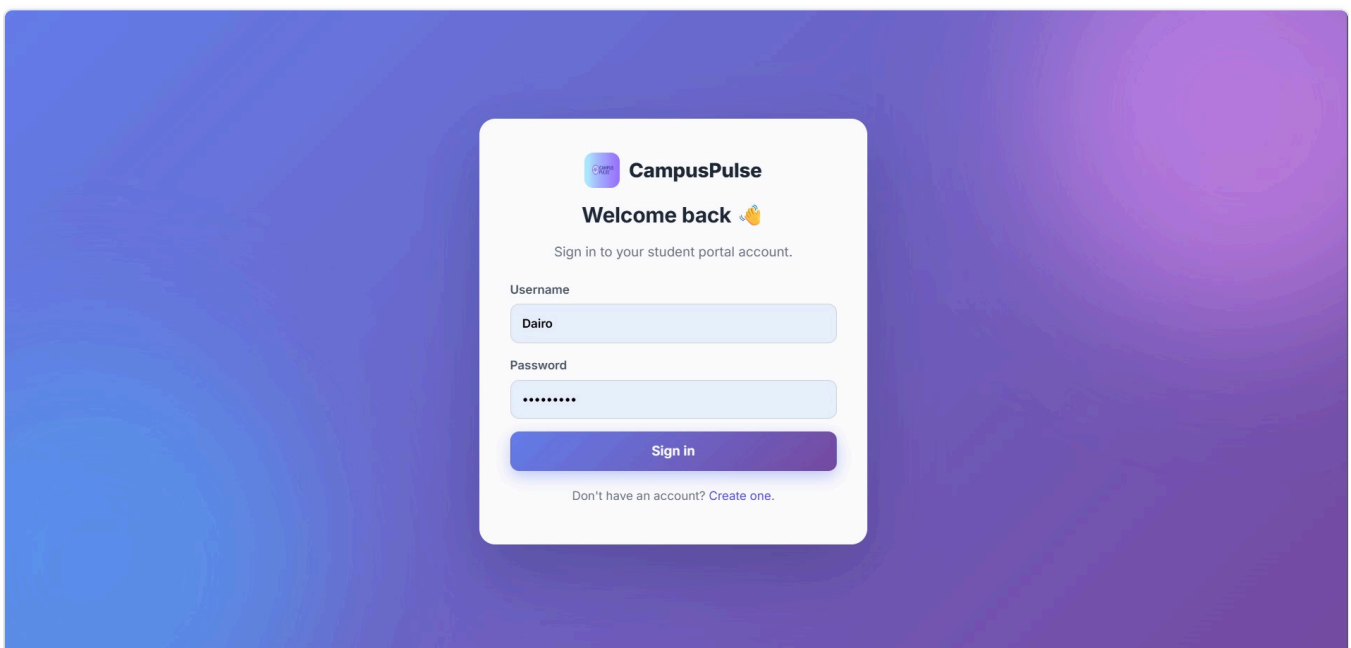
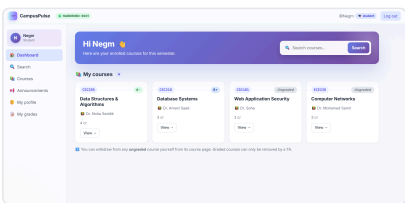


Figure 2.1 — The login page. Same screen on both builds; only the badge in the topbar changes.

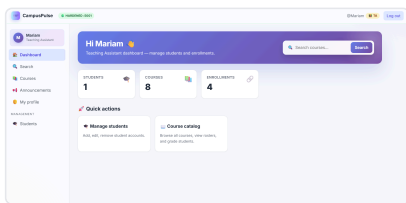
2.1 Three Roles, Three Views

The same `/dashboard` URL renders completely different content depending on who's logged in. That difference is enforced server-side — the UI hides what each role isn't allowed to do, but more importantly the backend rejects requests that bypass the UI.



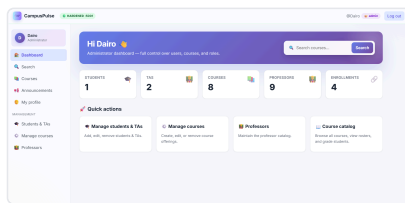
 **Student — Negm**

Sees only their own enrolled courses with live grade pills. Sidebar has *My profile* and *My grades* — no management links. Can self-withdraw from any *ungraded* course.



 **TA — Mariam & Nayrouz**

Stat cards (Students, Courses, Enrolments). Manages student enrolments and adds students to courses, but **cannot** assign grades or change roles — those are admin-only.

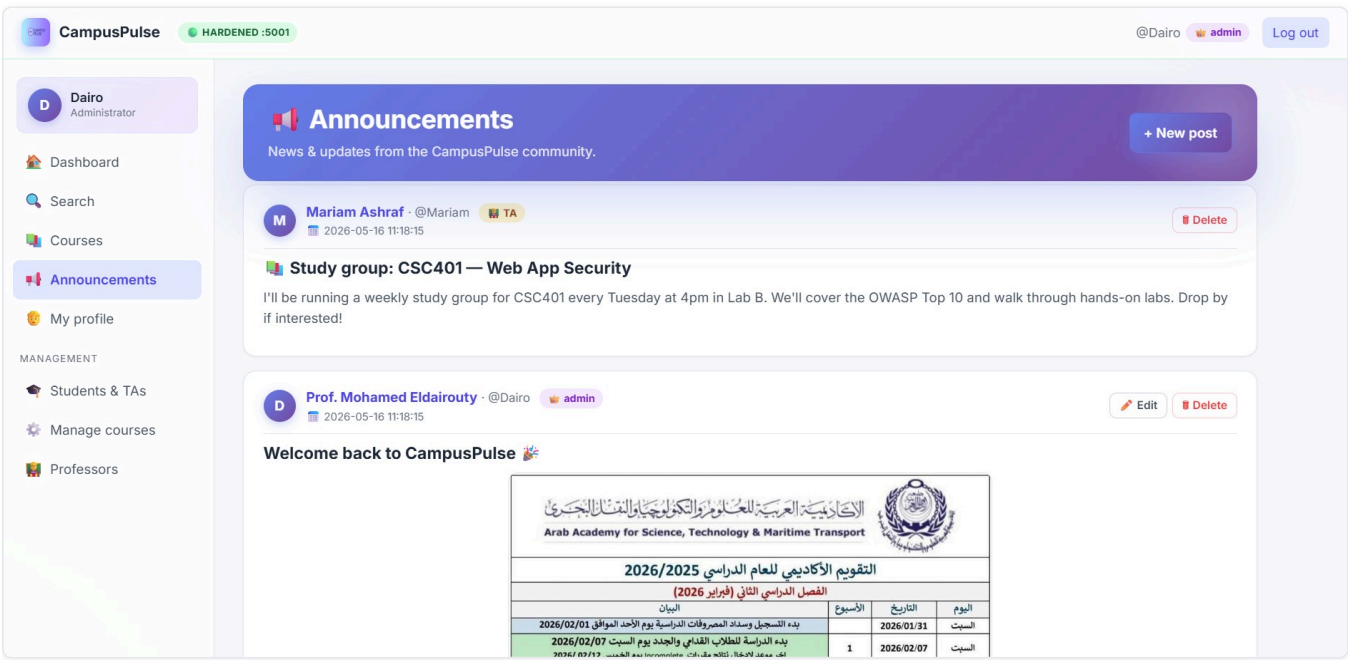


 **Admin — Dairo**

Full operational control: grade students, promote students to TA, manage courses and professors. The Promote/Demote buttons in the UI only ever offer the `ta` and `student` roles — never `admin`.

2.2 The Announcements Feed

The portal has a community feed where any user can post — news, study groups, exam schedule updates. It's harmless on its surface and looks like every other LMS noticeboard most universities run. Later we'll see that this feed becomes the perfect **delivery vector** for the chained attack.



التقويم الأكاديمي للعام الدراسي 2026/2025			
الفصل الدراسي الثاني (فبراير 2026)			
اليوم	التاريخ	الأسبوع	البيان
السبت	2026/01/31		بدء التسجيل وسداد المصروفات الدراسية - يوم الأحد الموالي
السبت	2026/02/07	1	بدء الدراسة للطلاب القاديين والجدد يوم السبت
	2026/02/07/19		انقضاء الإحقة - ثلاثة أيام - امتحان التقييمية مع المحيد

Figure 2.2 — Announcements feed with seed posts from the admin and a TA. Anyone with an account can publish here.

Architecture

3.1 Two-State Deployment

The hardest design decision early on was: do we run one app with a feature-flag switching between vulnerable and patched, or two independent apps? We chose **two independent apps**, identical in features and UI, that listen on different ports. Reviewers can `diff vulnerable/ mitigated/` and see exactly which lines moved from unsafe to safe — no hidden if-branches.

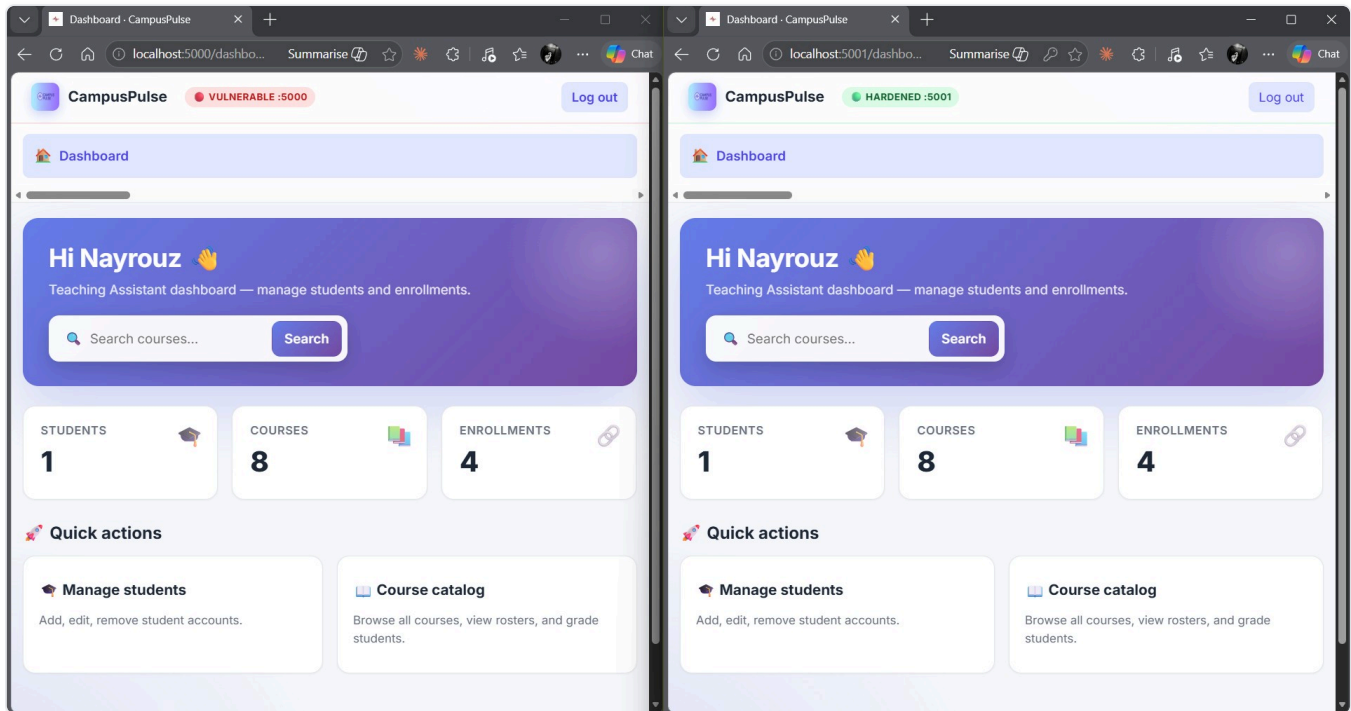


Figure 3.1 — Both deployments running side-by-side. Identical UI; only the topbar badge changes (● :5000 vs ● :5001).

3.2 Technology Stack

Layer	Choice	Why this and not something else
Web framework	Flask 3	Tiny, transparent — every route is one short function with no hidden middleware magic.
Templating	Jinja2 (server-rendered)	A React frontend would auto-escape everything; the reflected XSS would be impossible to demonstrate honestly.
Database	SQLite + raw <code>sqlite3</code>	An ORM auto-parameterises queries — SQLi can't even occur. We need raw SQL so the vulnerability is real and the fix is meaningful.
Sessions	Flask signed cookies, <code>SameSite=Lax</code>	CSRF requires cookie-based auth. JWT in <code>localStorage</code> can't be CSRF'd in the first place.
Passwords	Werkzeug PBKDF2	Even the vulnerable build hashes passwords — the project is about the chained vulnerabilities we plant, not about teaching plain-text storage.

3.3 Database Schema

Six tables, regenerated by a single `seed.py` script:

```
users          - accounts (student | ta | admin) + bio, major, year, stored GPA for TAs
professors     - instructor pool
courses       - course catalogue (code, title, description, credits)
course_professors - many-to-many: a course can be co-taught by multiple professors
enrollments   - student ↔ course with the specific lecturer + a grade (NULL = ungraded)
announcements  - community feed (title, body, image, optional click-through link)
```

The Chained Attack

The attacker is **Negm** — a regular student with no special privileges. Their goal is one of two things, achieved without ever guessing the admin's password:

1. 👑 **Privilege escalation** — promote themselves from student to admin (or TA).
2. 🤫 **Quiet grade tampering** — assign themselves an A+ in a course they're enrolled in.

Both goals use the **same chain**. Only the final POST URL differs.

4.1 Stage 1 — Reconnaissance via SQL Injection

Negm starts by abusing the course search bar. The vulnerable build constructs its query by f-string concatenation:

```
sql = ("SELECT c.id, c.code, c.title, ..., c.credits "
      "FROM courses c "
      f"WHERE c.code LIKE '%{q}%' OR c.title LIKE '%{q}%'"
      rows = db.execute(sql).fetchall()
```

The single quote in `'%{q}%'` is the door. Negm pastes a UNION payload into the search bar:

```
' UNION SELECT id, username, email, role, 1 FROM users--
```

The UNION half matches the 5-column shape of the original SELECT, so SQLite happily glues rows from the `users` table onto the (empty) course matches. The page renders the leaked rows as if they were ordinary course cards — exposing every account's `id`, `username`, `email`, and `role` in one shot.

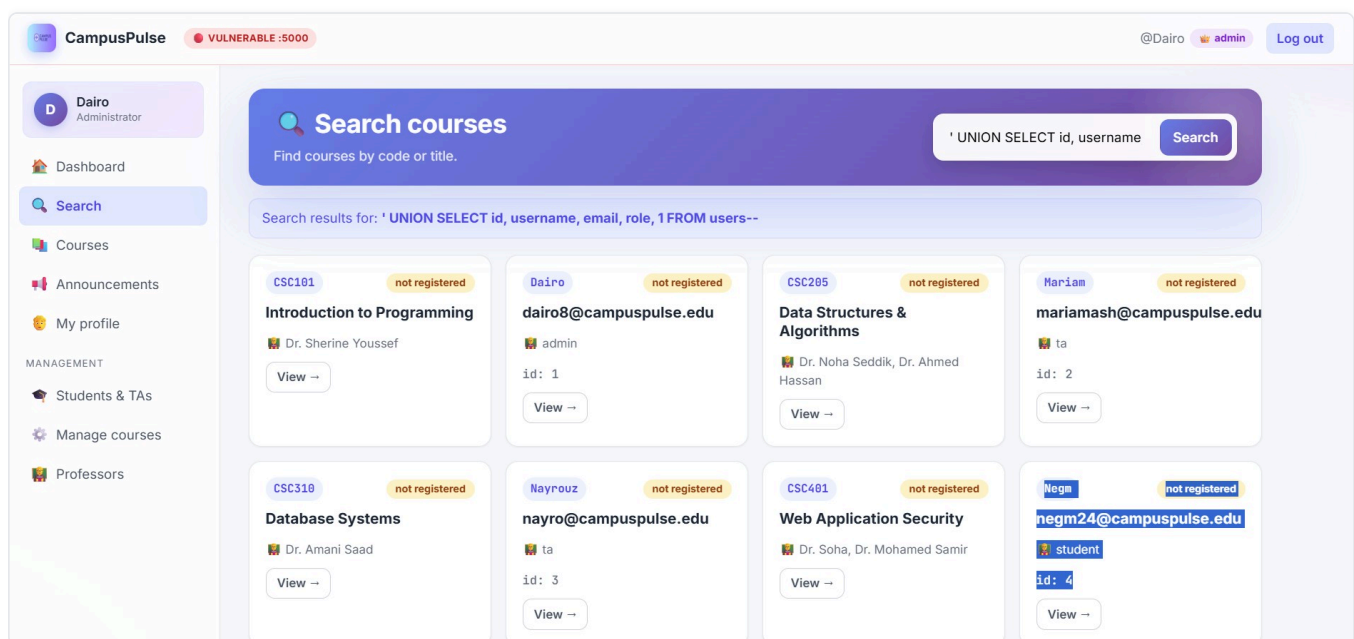


Figure 4.1 — One injection, an entire table exfiltrated. From here Negm learns his own `user_id = 4`, the admin's identity (Dairo, id 1), and everyone else's email addresses. Course IDs needed for Stage 3 are read straight off the `/courses` page URLs.

4.2 Stage 2 — Delivery via Reflected XSS

The search results page echoes the query string back through the template:

```
<p class="search-echo">
  Search results for: <strong>{{ q|safe }}</strong>
</p>
```

Jinja's default auto-escaping is bypassed by the `|safe` filter, so any HTML the attacker puts in `q` lands raw in the response. A crafted URL looks like this:

```
http://localhost:5000/search?q=<script>fetch('/courses/7/grade',
  {method:'POST', credentials:'include',
    body: new URLSearchParams({student_id: 4, grade: 'A+'})})</script>
```

Negm doesn't deliver this URL by emailing it — that would be unrealistic. Instead they post it on the very feed every staff member visits daily.

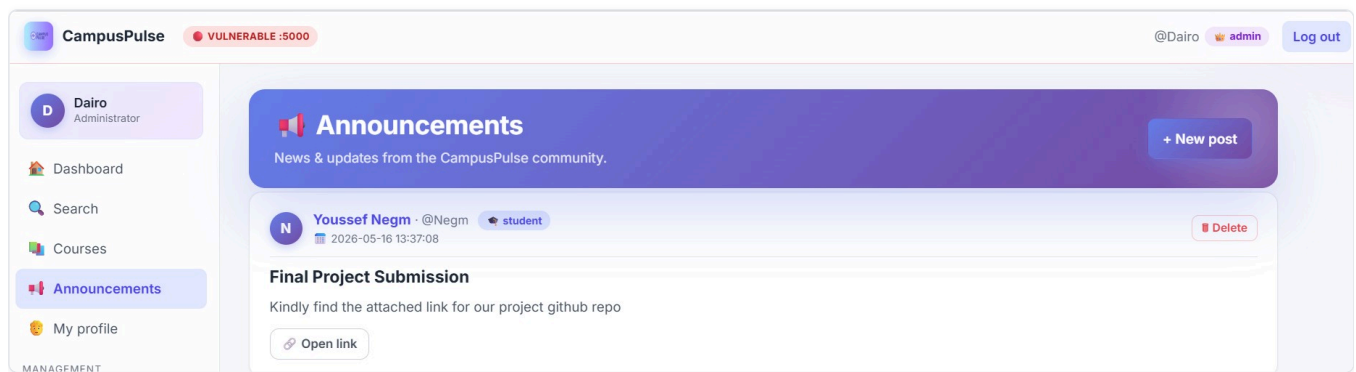


Figure 4.2 — The trap. Negm publishes an urgent-looking "Final exam schedule changed" announcement. The attached image looks like a real schedule poster; the whole thing is wrapped in a click-through link pointing at the crafted search URL.

4.3 Stage 3 — Execution via CSRF

The chain lies dormant until an admin loads the feed and clicks the announcement. The browser navigates to `/search?q=...`, the server echoes the `<script>` tag into the page, and the browser executes it *inside the admin's authenticated session*. The script issues a same-origin `fetch()` — and because `credentials:'include'` is set, the admin's session cookie travels with the request automatically.

Two things make this stage land:

1. The endpoint **has no CSRF token**: `POST /admin/users/set-role` (or `/courses/7/grade`) accepts any well-formed body and trusts the cookie.
2. For the privilege-escalation variant, the endpoint **has no role whitelist**: the UI normally only sends `target_role=ta` or `student`, but the server happily accepts `target_role=admin` from a forged request — a textbook mass-assignment flaw.

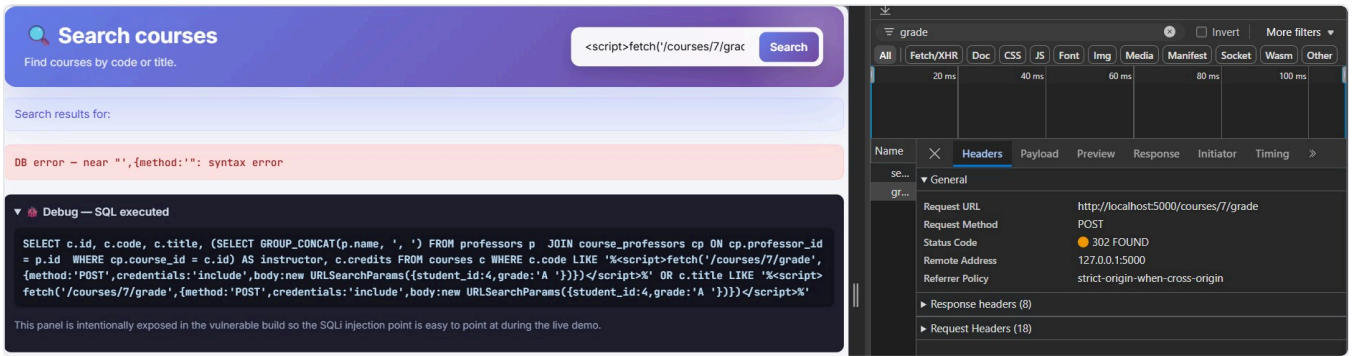


Figure 4.3 — DevTools Network tab in the admin's browser; the moment the announcement is clicked. The GET to `/search` is immediately followed by the silent POST that does the damage — returning HTTP 302 from a server that suspects nothing.

4.4 The Damage

Negm refreshes their own page and discovers either an A+ in CSC401 (the quiet variant) or the admin sidebar suddenly appearing in their account (the loud variant). The database is the source of truth — and the database has been modified.

vulnerable > **campuspulse.db**

Filter 7 t Rows: 4 Filter 4 rows...

us...	co...	profe...	grade
announcements	4	2	A-
course_professors	4	3	B+
courses	4	4	NULL
enrollments	4	7	A+
professors	4	7	A+
sqlite_sequence	4	7	A+
users	5		

Figure 4.4 — The DB after the chain fires. Either Negm's `users.role` is now `admin`, or his `enrollments.grade` for one course is now `A+` — depending on which final URL the attacker chose. Same chain, two different outcomes.

Why grade tampering is the sneakier outcome: from an audit log it looks like "the admin graded Negm A+ today" — perfectly ordinary admin activity. No suspicious role change, no UI fingerprint. The student just has a better grade.

Mitigations & the Logic Behind Them

Three flaws, three fixes. The hardened build receives every byte of every attack we just demonstrated — and rejects each one. Below we walk through each mitigation, what it does, and *why* it works.

5.1 Fix #1 — Parameterised Queries (SQL Injection)

The single line of code that ends SQL injection: stop building queries with string concatenation and start binding parameters.

● **VULNERABLE** (`VULNERABLE/APP/ROUTES.PY`)

```
sql = (
    "SELECT ... FROM courses c "
    f"WHERE c.code LIKE '%{q}%' "
    f"OR c.title LIKE '%{q}%'"
)
rows = db.execute(sql).fetchall()
```

● **HARDENED** (`MITIGATED/APP/ROUTES.PY`)

```
sql = (
    "SELECT ... FROM courses c "
    "WHERE c.code LIKE ? "
    "OR c.title LIKE ?"
)
like = f"%{q}%"
rows = db.execute(sql, (like, like)).fetchall()
```

The difference: in the hardened version, `q` never enters the SQL string. The SQLite driver passes it to the engine as a typed parameter — purely data, never syntax. The UNION payload that broke out of the LIKE clause in the vulnerable build is now just a literal text query, matched against course codes, and rightly returns zero rows.

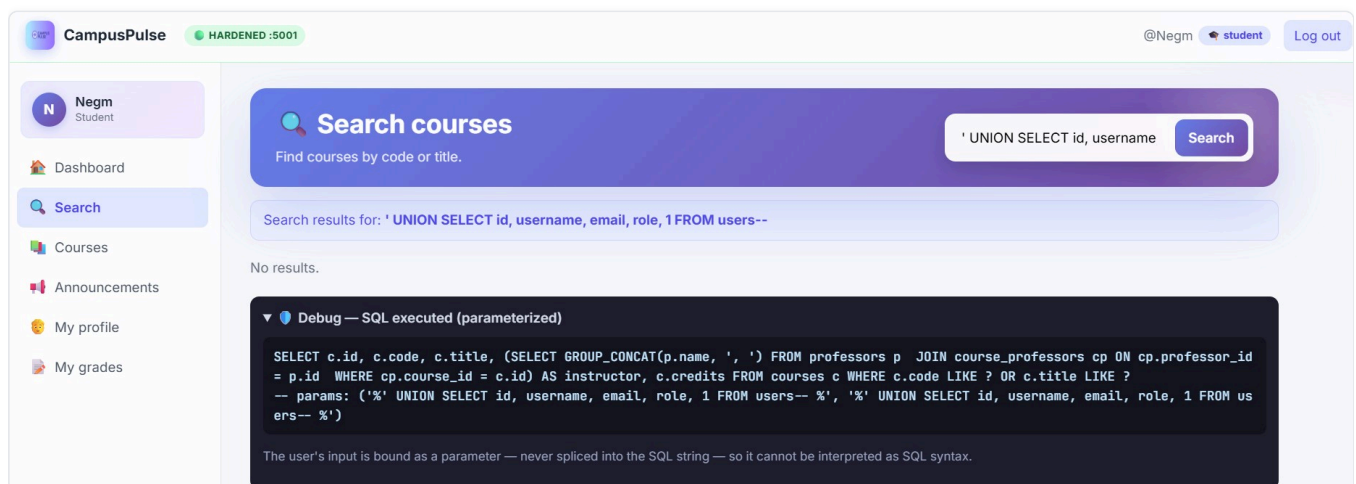


Figure 5.1 — The same UNION payload pasted into the hardened search. No user rows leaked. The Debug-SQL panel reveals the parameterised query and confirms the input is bound, not spliced.

5.2 Fix #2 — Output Encoding (Reflected XSS)

Jinja2 already auto-escapes `{{ q }}` by default. The vulnerable build actively disabled that protection with `|safe`. The fix is to remove that one filter — and, for defence in depth, set a strict **Content-Security-Policy** header that would block any `<script>` tag even if encoding ever failed.

VULNERABLE (`SEARCH.HTML`)

```
Search results for:
<strong>{{ q|safe }}</strong>
```

HARDENED

```
Search results for:
<strong>{{ q }}</strong>

+ CSP header:
default-src 'self';
script-src 'self';
object-src 'none';
```

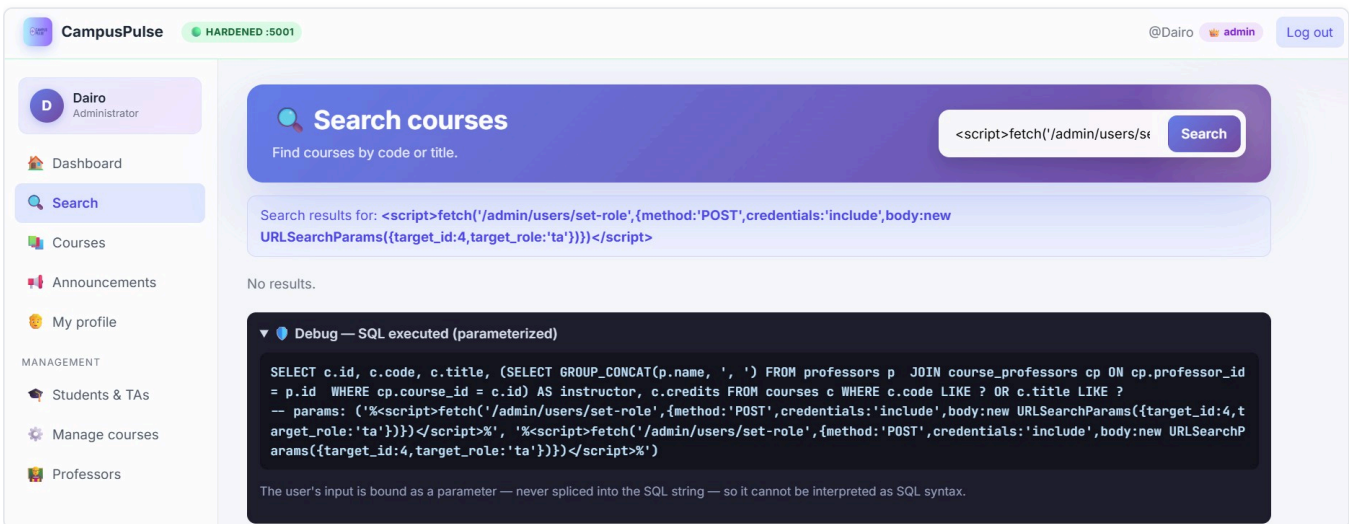


Figure 5.2 — The exact same set-role `<script>fetch(...)</script>` payload Negm used in Section 4 lands harmlessly on the green build — printed back as plain text instead of executing. Every angle bracket and quote is HTML-encoded (`<` , `>` , `'`) so the browser parses the whole thing as a string, never as markup.

5.3 Fix #3 — CSRF Token + Role Whitelist

Two complementary defences for the third stage. The token kills the chain entirely; the whitelist is defence-in-depth that closes the specific privilege-escalation path even if a token ever leaked.

(a) Per-session CSRF token. Every state-changing form embeds a hidden input whose value is a 32-byte random token tied to the user's session. A global `before_request` handler in `__init__.py` verifies the token on every non-GET request:

```
@app.before_request
def csrf_protect():
    if request.method in {"GET", "HEAD", "OPTIONS"}:
        return
    sent = request.form.get("csrf_token") or request.headers.get("X-CSRF-Token", "")
    expected = session.get("csrf_token", "")
    if not expected or not secrets.compare_digest(sent, expected):
        abort(400, description="CSRF token missing or invalid.")
```

The XSS payload that lands inside the admin's browser cannot read this token — it lives in the session, not in any DOM the script can access. So the forged POST arrives without a token, hits the `abort(400)`, and gets rejected.

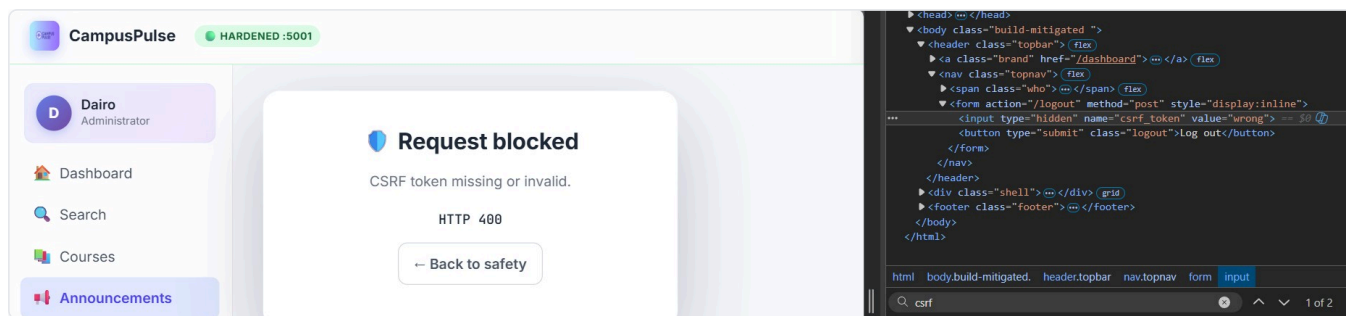


Figure 5.3 — A tokenless POST hits the 400 page on the hardened build. The same XSS payload that succeeded on port 5000 dies here before reaching the route handler.

(b) Server-side role whitelist. For the role-setting endpoint specifically, we also added an allow-list of valid `target_role` values — so that even if an attacker somehow obtained a valid CSRF token, the request to escalate to `admin` would still be rejected:

```
ALLOWED_ROLES_FOR_SET_ROLE = {"student", "ta"} # never includes "admin"

@app.route("/admin/users/set-role", methods=["POST"])
@role_required("admin")
def admin_user_set_role():
    target_role = (request.form.get("target_role") or "").strip()
    if target_role not in ALLOWED_ROLES_FOR_SET_ROLE:
        flash("Invalid request – role must be 'student' or 'ta'.", "error")
        return redirect(url_for("main.admin_users"))
    # ... safe to write
```

This is the "principle of least privilege" applied to a single form field. The UI never offers `admin` as an option, so the server has no business accepting it either.

5.4 Mitigation Summary

Stage	Vulnerability	Fix	Why it works
1	SQL Injection	Parameterised placeholders	User input is sent to the engine as data, never compiled as syntax.
2	Reflected XSS	Drop <code> safe</code> + strict CSP	HTML-encoding turns <code><script></code> into harmless text; CSP forbids inline scripts even if encoding ever fails.
3	CSRF	Per-session token + <code>SameSite=Lax</code> cookie	The forged request can't supply the token; the cookie won't accompany cross-site POSTs anyway.
3b	Mass-assignment	Role whitelist <code>{student, ta}</code>	Even with a valid token, the server refuses to grant the one role the UI never offered.

Tools We Used

Category	Tool	Role in this project
Editor	Visual Studio Code	Day-to-day coding for both builds.
Database inspection	SQLite Viewer (VS Code extension, <code>qwtel.sqlite-viewer</code>)	Watching <code>users.role</code> and <code>enrollments.grade</code> flip live during the attack.
HTTP inspection	Browser DevTools (Network, Elements, Application)	Confirming the forged POST fired and inspecting the Content-Security-Policy header.
Quick scripting	<code>curl</code>	Reproducing each stage of the attack from the command line for regression-checking the mitigations.
Source control	Git + GitHub	Repository hosted at github.com/MohamedEldairouty/CampusPulse-Shield .

Reflections

The original CampusPulse — built by (Youssef-Nayrouz) for the Web Development course — is a working student portal that we used as our visual reference and starting point. We forked the idea, rebuilt the architecture in a way that lets the security demo be honest (server-rendered templates, raw SQL, cookie-based sessions), and ended up with something we ourselves would be wary of deploying without the green-build mitigations in place.

The repository, the live demo recording, this report, and the attack cheatsheet are all available in github.com/MohamedEldairouty/CampusPulse-Shield.